

Query Optimization: Nested Subqueries

1 Introduction

SQL conceptually treats nested subqueries in the where clause as functions that take parameters and return either a single value or a set of values.

Correlation Variables

The parameters passed to a subquery from the outer query are called **correlation variables**.

Subquery as a Function

Find instructors who taught in 2019:

```
select name from instructor
where exists (select * from teaches
              where instructor.ID = teaches.ID
              and teaches.year = 2019);
```

The subquery takes `instructor.ID` as a parameter and returns the set of courses taught by that specific instructor in 2019.

2 Evaluation Strategies

2.1 Correlated Evaluation

Conceptually, the outer query computes a Cartesian product, and the subquery is executed **once for each tuple** generated.

Performance Inefficiency

Correlated evaluation is often slow because:

- A huge number of calls are made to the nested query.
- It causes excessive **random disk I/O** operations.

2.2 Decorrelation

Optimizers attempt **decorrelation**—the process of replacing a nested query with a join, semi-join, or anti-semijoin—to utilize efficient algorithms like Merge Join or Hash Join.

3 Multiset Semantics and Duplicates

A standard join like $\Pi_{name}(\text{instructor} \bowtie \text{teaches})$ is **incorrect** because SQL uses multiset semantics.

Join vs. Semijoin: The Duplicate Problem

If an instructor teaches 3 sections in 2019:

- **SQL Subquery:** Returns the instructor *once* (since `exists` is a boolean check).
- **Standard Join:** Returns the instructor *three* times (one for each matching `teaches` tuple).
- **Multiset Semijoin:** Returns the instructor *exactly as many times* as they appeared in the original instructor relation.

The Semijoin Solution

The multiset **semijoin** ($\ltimes$) operator solves this. If a tuple r_i appears n times in r , it appears n times in the result of $r \ltimes_{\theta} s$ if there is at least one tuple in s that satisfies θ ; otherwise, it is excluded.

The correct Relational Algebra is:

$$\Pi_{name}(\text{instructor} \ltimes_{\text{instructor.ID}=\text{teaches.ID} \wedge \text{teaches.year}=2019} \text{teaches})$$

4 Decorrelation Examples

4.1 EXISTS and IN Clauses

The following two SQL queries are equivalent and translate to the same semijoin expression.

Using IN Clause

```
select name from instructor
where instructor.ID in (select teaches.ID from teaches
                        where teaches.year = 2019);
```

Relational Algebra:

$$\Pi_{name}(\text{instructor} \ltimes_{\text{instructor.ID}=\text{teaches.ID}} (\sigma_{\text{year}=2019}(\text{teaches})))$$

4.2 NOT EXISTS and Anti-semijoin

For queries involving **negative checks**, the anti-semijoin is used.

Using NOT EXISTS

```
select name from instructor
where not exists (select * from teaches
                  where instructor.ID = teaches.ID
                  and teaches.year = 2019);
```

Relational Algebra (Anti-semijoin $\bar{\bowtie}$):

$$\Pi_{name}(\text{instructor} \bar{\bowtie}_{\text{instructor.ID} = \text{teaches.ID}} (\sigma_{year=2019}(\text{teaches})))$$

5 General Transformation Rule

A generic SQL query of the form:

Generic Decorrelation Rule

Input SQL:

```
select A from r1, r2, ..., rn
where P1 and exists (select * from s1, s2, ..., sm
                    where P21 and P22)
```

Output Relational Algebra:

$$\Pi_A(\sigma_{P_1}(r_1 \times \cdots \times r_n) \bowtie_{P_{22}} \sigma_{P_{21}}(s_1 \times \cdots \times s_m))$$

- P_{21} : Predicates referencing only relations s_i in the subquery.
- P_{22} : Predicates referencing correlation variables from relations r_i in the outer query.

6 Complex Decorrelation: Aggregation

Decorrelation is more complex when subqueries use aggregation or are used as scalar values.

Scalar Aggregate Subquery

Find instructors who taught more than one section in 2019:

```
select name from instructor
where 1 < (select count(*) from teaches
          where instructor.ID = teaches.ID
          and teaches.year = 2019);
```

Refined Relational Algebra:

$$\Pi_{name}(\text{instructor} \bowtie_{\substack{\text{ID}=\text{TID} \\ \wedge 1 < \text{cnt}}} \text{ID as TID} \gamma_{\text{count(*) as cnt}} (\sigma_{year=2019}(\text{teaches})))$$

Risks and Limitations

- **Scalar Safety:** Decorrelated queries might return results if the subquery returns multiple values, whereas SQL would throw a runtime error.
- **Heuristic vs. Cost-based:** Sophisticated optimizers only decorrelate if it reduces the estimated cost.